

Data Structures and Algorithms I

Assignment 1: Arrays

Complexity Analysis

Operation	Time	Space	Justification
Most Frequent Registrant	$O(n)$	$O(n)$	$O(n)$ — list is scanned twice (once for counting, once for checking max). $O(n)$ — dictionary stores all unique IDs.
Ordered Deduplication	$O(n)$	$O(n)$	$O(n)$ — single pass through list with constant-time set operations. $O(n)$ — set and result list may store all unique IDs.
First Unique Student	$O(n)$	$O(n)$	$O(n)$ — two separate passes over the data. $O(n)$ — dictionary stores frequency of IDs.
Contiguous Subarray Sum	$O(n)$	$O(n)$	$O(n)$ — single pass using prefix sums. $O(n)$ — set stores prefix sums.

Assignment 2: Linked Lists

Complexity Analysis

Operation	Time	Space	Justification
Enqueue Ticket	$O(n)$	$O(1)$	$O(n)$ — traverse to end of list. Space: $O(1)$ — no extra memory used.
Priority Insert	$O(n)$	$O(1)$	$O(n)$ — search for position. Space: $O(1)$ — only one new node added.
Resolve Ticket	$O(n)$	$O(1)$	Time: $O(n)$ — search then delete. Space: $O(1)$ — only pointer changes.
Find Middle Ticket	$O(n)$	$O(1)$	$O(n)$ — single traversal using fast/slow pointers. $O(1)$ — only two pointers used.
Reverse Queue	$O(n)$	$O(1)$	Time: $O(n)$ — visit each node once. Space: $O(1)$ — in-place reversal.

Operation	Time	Space	Justification
Display Queue	$O(n)$	$O(1)$	$O(n)$ — traverse all nodes. $O(1)$ — no extra storage.

Bonus Challenge: Array vs. Linked List

1. Search Complexity:

Singly Linked List: $O(n)$

Python List (Array): $O(1)$ for indexing and faster sequential searches due to memory locality.

2. CPU Cache Locality:

Python lists store elements in contiguous memory locations. This improves cache performance because the CPU can load nearby elements efficiently.

Linked lists store nodes in different memory locations. Traversing them requires following pointers, causing cache misses and slower execution.

3. Memory Overhead

Each linked list node stores:

- Ticket data
- A pointer (next)

This extra pointer increases memory usage.

Python lists only store references in contiguous memory and therefore have lower overhead per element.